INTRODUCTION TO SQL (Structured Query Language)

- The name SQL is derived from Structured Query Language

- Originally called as SEQUEL (Structured English QUEry Language), designed and implemented at IBM Research

- SQL is now the standard language for commercial relational DBMSs.

The Example Database

The database which has the four tables S, P, J, SPJ is considered here.

Oracle Database Express Edition 10*g* Release 2 (10.2)

Data Definition Language

- SQL uses the terms, *table*, *row*, and *column* for the formal relational model terms ***relation, tuple,*** and ***attribute.***

Data Definition Language

- CREATE TABLE
- ALTER TABLE
- DROP TABLE
- CREATE VIEW
- DROP VIEW

Creating Tables

- The CREATE TABLE command is used to specify a new relation by giving
  - A name
  - Its attributes and initial constraints
- Each attribute has
  - A name
  - A data type to specify its domain values and
  - Any attribute constraints such as NOT NULL

Creating Tables

- The key, entity integrity and referential integrity constraints can be specified within the create statement after the attributes are declared, or they can be added later using the ALTER table command.

- The relations created through CREATE statement are called **base tables**.

- This means the relations and its tuples are actually created and stored as a file by the DBMS.

Creating Tables

To create the table **S**:

```
CREATE TABLE S
  (SNO CHAR(2),
   SNAME VARCHAR(20),
   STATUS NUMBER,
   CITY VARCHAR(15));
```

Note the **;** at the end of the statement

Here key, entity integrity and referential integrity constraints are not specified. This is a basic create statement. We will look later to include the constraints

To create the table PART:

```
CREATE TABLE  P
(Pno CHAR(2),
PNAME  VARCHAR(20),
PRICE   NUMBER,
WEIGHT NUMBER,
CITY   VARCHAR(15));
```

P				
PNO	**PNAME**	**PRICE**	**WEIGHT**	**CITY**

To create the table SUPPLIER:

```
CREATE TABLE  S
(SNO CHAR(2),
SNAME  VARCHAR(20),
STATUS   NUMBER,
CITY   VARCHAR(15));
```

S			
SNO	**SNAME**	**STATUS**	**CITY**

S			
SNO	SNAME	STATUS	CITY

To ALTER the table SUPPLIER by adding Column:

S				
SNO	SNAME	STATUS	CITY	COUNTRY

ALTER TABLE S
ADD COUNTRY VARCHAR2(50);

To ALTER the table SUPPLIER by removing a Column:

S		
SNO	SNAME	STATUS

ALTER TABLE S
DROP COLUMN CITY;

S			
SNO	**SNAME**	**STATUS**	**CITY**

To DROP the table SUPPLIER:

DROP TABLE S;

Data Manipulation Language

The principal DML statements in SQL are:

- INSERT
- SELECT
- DELETE
- UPDATE

Inserting Tuples

INSERT INTO **S** VALUES

(‘S6’, ‘GAMUNU', 20, ‘COLOMBO');

Note the single quotes around the character data

CHAR(2)

S			
SNO	**SNAME**	**STATUS**	**CITY**
S1	SAMAN	20	COLOMBO
S2	JAGATH	10	KANDY
S3	ANIL	30	KANDY
S4	GAMINI	20	COLOMBO
S5	BALA	30	JAFFNA
S6	GAMUNU	20	COLOMBO

Simple Retrieval

- SELECT command is used to retrieve data from the database.
- SELECT command returns a result set in the form of a table, referring to one or more tables and applying required conditions as per the requirements.

General form of SELECT

```
SELECT [ DISTINCT ] attribute(s)
FROM   table(s)
[WHERE condition]
[GROUP BY attribute(s)]
[HAVING condition]
[ORDER BY attribute(s)]
```

Get the order right!

Simple Retrieval

Get part numbers for all parts.

```
SELECT Pno
FROM P;
```

PNO
P1
P2
P3
P4
P5
P6

Get the supplier number, name, status and location of all suppliers

```
SELECT *
FROM S;

SELECT Sno, SNAME, STATUS,CITY
FROM S;
```

S			
SNO	**SNAME**	**STATUS**	**CITY**
S1	SAMAN	20	COLOMBO
S2	JAGATH	10	KANDY
S3	ANIL	30	KANDY
S4	GAMINI	20	COLOMBO
S5	BALA	30	JAFFNA
S6		20	COLOMBO

Removing Duplicates

Get the part numbers for all parts currently being supplied.

```
SELECT DISTINCT Pno
FROM SPJ;
```

PNO
P1
P2
P3
P4
P5
P6

QTY
200
700
200
200
500
600
400
800
100
200
500
300
300
1000
200
100
1200
800
400
500
100
200
500

PNO
P1
P2
P3
P3
P3
P3
P3
P3
P5
P3
P4
P2
P2
P1
P2
P2
P3
P4
P5
P5
P5
P6
P6

Qualified Retrieval

Get supplier numbers for suppliers in Kandy with >20 status value

```
SELECT Sno
 FROM S
 WHERE CITY = 'KANDY'
 AND STATUS > 20;
```

SNO
S3

S		
SNO	**SNAME**	**STATU**
S1	SAMAN	
S2	JAGATH	
S3	**ANIL**	
S4	GAMINI	
S5	BALA	
S6		

Get suppliers whose location is **known**, i.e. **not null**

```
SELECT *
FROM S
WHERE CITY IS NOT NULL;
```

SNO	SNAME	STATUS
S1	SAMAN	20
S3	ANIL	30
S4	GAMINI	20
S5	BALA	30
S6		20

original	
STATUS	CITY
20	COLOMBO
10	
30	KANDY
20	COLOMBO
30	JAFFNA
20	COLOMBO

Find the parts which are located in Colombo and has weight greater than 15.

Select pno, pname, weight, city

From P

Where city = ‘Colombo’ and weight > 15;

PNO	PNAME	WEIGHT	CITY
P6	**COG**	**19**	**COLOMBO**

PNO	**PNA**
P1	NUT
P2	BOLT
P3	SCRE
P4	SCRE
P5	CAM
P6	**COG**

Find the suppliers who have status values between 10 and 20.

```
Select sname
From S
Where status >=10 and status <= 20;
```

S ori		
SNO	**SNAME**	
S1	SAMAN	
S2	JAGATH	
S3	ANIL	
S4	GAMINI	
S5	BALA	
S6		

```
Select sname
From S
Where status between 10 and 20;
```

SNAME
SAMAN
JAGATH
GAMINI

Get the details of suppliers who are either located in Colombo or in Jaffna

```
Select *
From S
Where city = 'COLOMBO' or city = 'KANDY';
```

```
Select *
From S
Where city in ( 'Colombo' , 'Kandy');
```

S ori		
SNO	**SNAME**	
S1	SAMAN	
S4	GAMINI	
S5	BALA	
S6		

For each part in Colombo get the name and its weight in Kilograms

```
Select pname, weight/1000 as kweight
From P
Where city = 'COLOMBO';
```

KWEIGHT
0.012
0.014
0.019

P					
PNO	**PNAME**	**PRICE**	**COLOR**	**WEIGHT**	**CITY**
P1	NUT	20.00	RED	12	COLOMBO
P2	BOLT	14.00	GREEN	17	KANDY
P3	SCREW	14.00	BLUE	17	MATARA

Retrieval with Ordering

Get the supplier number and the status for suppliers in Kandy, in descending order of the status

```
SELECT Sno, STATUS
FROM S
WHERE CITY = ‘KANDY'
ORDER BY STATUS DESC;
```

Note: The default order is in ascending order.

DESC – if we want the result in descending order

ASC - if we want the result in ascending order, this is optional

Substring Pattern Matching

Get all suppliers whose name has ‘A’ as its second character.

```
SELECT SNAME
FROM S
WHERE SNAME LIKE ‘?A*’;
```

* - replaces an arbitrary number of zero or more characters

? - replaces a single character

Note : in Oracle db use % in place of * and

_ In place of ?

Renaming of Attributes

```
SELECT SNAME AS SUP_NAME
FROM S
WHERE SNAME LIKE '_A%';
```

- The new name SUP_NAME will appear as column header in the query result.

Retrieve the part names that start with letter C

```
Select pname
From P
Where pname like ‘C%’;
```

Retrieve the surnames of suppliers whose name begin with any vowels.

```
Select sname
From S
Where regexp_like(sname, '^[A,E,I,O,U]');
```

Retrieve the surnames of suppliers whose name begin with any letter A to P.

```
Select sname
From S
Where regexp_like(sname, '^[A-P]');
```

Retrieve the surnames of suppliers who have an even number as the first character of their status.

```
Select sname
From S
Where regexp_like(status, '^[2,4,6,8]');
```

Using Aggregate Functions

A number of functions exists: COUNT, SUM, AVG, MAX, MIN

COUNT – returns the number of tuples or values specified in a query.

SUM, MAX, MIN, AVG – are applied to a set or multiset of numeric values and return, respectively the sum, the maximum value, the minimum value, and average (mean) of those values.

These functions can be used in the <u>SELECT</u> clause or the <u>HAVING</u> clause.

Get the total number of suppliers.

```
SELECT COUNT(*) AS NUM_OF_SUP
FROM S;
```

NUM_OF_SUP
5

Get the total number of suppliers currently supplying parts

```
SELECT COUNT (DISTINCT Sno) AS TOTAL
FROM SPJ;
```

Note : in Access db cannot use the keyword distinct as in the above query.

Find the maximum and minimum weight of parts.

Select max(weight) As maxweight, min(weight) as minweight
From P;

MAXWEIGHT	MINWEIGHT
19	12

P					
PNO	**PNAME**	**PRICE**	**COLOR**	**WEIGHT**	**CITY**
P1	NUT	20.00	RED	12	COLOMBO
P2	BOLT	14.00	GREEN	17	KANDY
P3	SCREW	14.00	BLUE	17	MATARA
P4	SCREW	11.00	RED	14	COLOMBO
P5	CAM	150.00	BLUE	12	KANDY
P6	COG	12.00	RED	19	COLOMBO

Find the number of shipments for part P2.

Select count(*)

From SPJ

Where pno= ‘P2’;

COUNT(*)
5

SPJ			
SNO	PNO	JNO	QTY
S1	P1	J1	200
S1	P2	J4	700
S2	P3	J2	200
S2	P3	J3	200
S2	P3	J4	500
S2	P3	J5	600
S2	P3	J6	400
S2	P3	J7	800
S2	P5	J2	100
S3	P3	J3	200
S3	P4	J4	500
S4	P2	J1	300
S4	P2	J4	300
S5	P1	J2	1000
S5	P2	J2	200
S5	P2	J4	100
S5	P3	J4	1200
S5	P4	J4	800
S5	P5	J4	400
S5	P5	J5	500
S5	P5	J7	100
S5	P6	J2	200
S5	P6	J4	500

Find the average price of parts located in Colombo.

```
Select avg(price) as aprice
From P
Where city = ‘COLOMBO’;
```

APRICE
14.333333333333333333333333333333333333

Get the average price of the parts which are located in all cities other than ‘Matara’.

```
Select avg(price) as avgprice
From P
Where city <> ‘Matara’;

Where not city = ‘Matara’;

Where city not in (‘Matara’);
```

AVGPRICE
36.8333333333333333333333333333333333333

Group by clause

- Rearranges the table into partitions or groups, s.t. within any one group all rows have the same value for the group by field.
- The column in the select clause must have a single value for each group in the result.
- No other columns may be used in the select clause except those that are listed after the group by and the column that contains the result of the aggregate function.

For each part currently being supplied, get the part number and the total quantity supplied for that part

```
SELECT Pno, SUM(QTY)
FROM SPJ
GROUP BY Pno;
```

Comment on the following :

```
SELECT Pno, Sno, SUM(QTY)
FROM SPJ
GROUP BY Pno;
```

********** invalid group by expression

********** to correct we can add Sno also in to Group by clause

For each supplier and each part, get the total quantity supplied of this part by this supplier

```
SELECT  Sno, Pno, SUM(QTY)
FROM SPJ
GROUP BY Sno, Pno;
```

For each part supplied, get the number of suppliers currently supplying that part

```
SELECT  Pno, COUNT(DISTINCT Sno) AS NUM FROM SPJ
GROUP BY Pno;
```

For each part supplied, get the number of shipments of that part

```
SELECT  Pno, COUNT(*) AS NUM
FROM SPJ
GROUP BY Pno;
```

For each supplier and each part, get the number of projects that the supplier is currently supplying the part to

```
SELECT  Sno, Pno, COUNT(Jno)
FROM SPJ
GROUP BY Sno, Pno;
```

- **Where** clause
  - Used to eliminate rows
- **Having** clause
  - Used to eliminate groups

Get the part numbers for all parts supplied in more than one shipment

```
SELECT Pno
FROM SPJ
GROUP BY Pno
HAVING COUNT(*) > 1;
```

Calculate the average weight of the parts located in Colombo and located in Kandy.

```
Select city, avg(weight)
From P
Group by city
Having city in (‘Colombo’,’Kandy);
```

For each part s.t. the total quantity supplied is greater than 1000, get the part number and the maximum quantity of that part supplied.

```
Select pno, max(qty)
From SPJ
Group by pno
Having sum(qty) > 1000;
```

For the above query, order the result in increasing quantity value.

```
Select pno, max(qty)
From SPJ
Group by pno
Having sum(qty) > 1000
Order by 2;
```

For the above query exclude all the shipments which has quantity less than 200.

```
Select pno, max(qty)
From SPJ
Where qty > 200
Group by pno
Having sum(qty) > 1000
Order by 2;
```